

UNIVERSIDAD DEL VALLE DE GUATEMALA

FACULTAD DE INGENIERÍA

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CC3086 — Computación Paralela y Distribuida

PROYECTO NO. 1

Optimización y Paralelización con OpenMP de un Simulador de Fuegos Artificiales en SDL2

Evaluación de Escalabilidad, Impacto del Techo de Amdahl y Agrupamiento de Primitivas Gráficas

Integrantes / Autores:	• Jonathan (Zacatac23) — Carné: 231104 • June Herrera (skemono) — Carné: 231038 • Hugo Barillas [Hb_W] — Carné: 23306
Curso:	CC3086 — Computación Paralela y Distribuida
Catedrático:	Equipo Docente de Computación Paralela y Distribuida
Fecha de Entrega:	Agosto de 2024 / 2026
Entorno de Ejecución:	GCC 16.2.0 (UCRT64 / MinGW-w64) / OpenMP 4.5 / SDL2 2.32.10

Índice General

1. Resumen Ejecutivo / Abstract	2
2. Introducción	3
3. Antecedentes y Marco Teórico	3
3.1 Modelo de Memoria Compartida y OpenMP	3
3.2 Ley de Amdahl y el Techo de Rendimiento	3
3.3 Sobrecarga de Hilos, Sincronización y <i>False Sharing</i>	4
3.4 Generación Pseudoaleatoria Concurrente (PRNG)	4
4. Metodología e Implementación Técnica	4
4.1 Arquitectura del Sistema y Modelado de Partículas	4
4.2 Estrategia de Paralelización con OpenMP	5
4.3 Rediseño del Pipeline Gráfico: Agrupamiento en Cubos (Batching)	5
4.4 Programación Defensiva y Cláusulas Condicionales	6
4.5 Metodología Experimental y Mitigación del Sesgo Térmico	6
5. Análisis de Resultados y Discusión	7
5.1 Escalabilidad Fuerte (Strong Scaling) con Hilos	7
5.2 Escalabilidad con el Tamaño del Problema (N)	8
5.3 Demostración del Techo de Amdahl y Comparativa de Render	8
6. Conclusiones y Recomendaciones	9
7. Referencias Bibliográficas	10
8. Anexo 1 — Diagrama de Flujo del Programa	11
9. Anexo 2 — Catálogo de Funciones	12
10. Anexo 3 — Bitácora de Pruebas y Mediciones Experimentales	15

Resumen Ejecutivo

El presente informe documenta el diseño, implementación, optimización y evaluación experimental de un simulador de partículas en tiempo real (fuegos artificiales) desarrollado en lenguaje C estándar (C11) con la biblioteca multimedia SDL2 y paralelizado mediante OpenMP. Se analiza en profundidad la partición del trabajo computacional en memoria compartida, la eliminación de cuellos de botella seriales mediante el rediseño del pipeline gráfico (batching con `SDL_RenderDrawPoints`), la mitigación de *false sharing* en estructuras de control concurrentes y generadores pseudoaleatorios, y la validación de la **Ley de Amdahl**. Los resultados experimentales demuestran un *speedup* de cómputo puro de hasta **3.74x** con 16 hilos a $N = 200,000$ partículas, y un incremento superior a **11.5x en los FPS globales** tras la optimización del renderizado por lotes, elevando la fracción paralelizable del frame del 2.9% a más del 67.9% y eliminando la saturación artificial impuesta por el API gráfico.

1. Introducción

La simulación de sistemas de partículas constituye un problema clásico y representativo en la computación de alto rendimiento (HPC). Cada partícula sigue ecuaciones de movimiento newtoniano discretizado (posición, velocidad, gravedad, resistencia aerodinámica y decaimiento de vida) que son inherentemente independientes entre sí, convirtiendo a la fase de física en un problema *vergonzosamente paralelo* (embarrassingly parallel). Sin embargo, en aplicaciones interactivas con despliegue visual inmediato, el rendimiento global no depende únicamente de la velocidad con que se integran las ecuaciones físicas, sino también del costo de transferir y dibujar dichas entidades en pantalla.

El objetivo fundamental de este proyecto es transformar una aplicación base 100% secuencial en una solución paralela de alto desempeño utilizando OpenMP bajo una arquitectura multinúcleo x86_64, evaluando cuantitativamente las métricas de **Speedup (\$\$)**, **Eficiencia (\$E\$)** y el **Techo de Amdahl**. Asimismo, se aborda el fenómeno donde el subsistema gráfico mono-hilo de SDL2 domina el tiempo de ejecución, demostrando que la verdadera optimización paralela requiere reestructurar tanto el cómputo como la forma en que los datos se preparan para su consumo serial.

2. Antecedentes y Marco Teórico

2.1 Computación Paralela y Modelo de Memoria Compartida

OpenMP (*Open Multi-Processing*) es el estándar de facto para programación concurrente en arquitecturas de memoria compartida. Basado en el modelo *Fork-Join*, la ejecución inicia en un hilo maestro (hilo 0) y, al encontrar directivas estructuradas como `#pragma omp parallel`, se crea o despierta un equipo de hilos trabajadores que ejecutan concurrentemente las regiones designadas [1]. La distribución de iteraciones en bucles cerrados se gestiona mediante directivas `#pragma omp for` con esquemas de planificación **static** (reparto equitativo precalculado, óptimo para cargas homogéneas) o **dynamic** (asignación dinámica por bloques de tamaño *chunk*, ideal para cargas con balanceo irregular o condiciones de salida temprana) [2].

2.2 Ley de Amdahl y el Techo Teórico de Rendimiento

Formulada por Gene Amdahl en 1967 [3], la ley establece que la aceleración máxima alcanzable por un programa paralelizado sobre p procesadores está acotada por la fracción serial intrínseca ($1 - f$) que no puede ser paralelizada:

$$\text{Ley de Amdahl: } S(p) = 1 / [(1 - f) + (f / p)] \quad \text{Límite Asintótico } (p \rightarrow \infty): S_{\text{máx}} = 1 / (1 - f)$$

Donde f es la fracción del tiempo de ejecución total susceptible de paralelización y p es el número de hilos.

En un simulador gráfico, si el renderizado toma el 95% del frame de manera serial ($f = 0.05$), el speedup global jamás superará $1 / (1 - 0.05) = 1.052 \times$, incluso disponiendo de infinitos procesadores. Para que el paralelismo tenga impacto real en los FPS, es imperativo reducir el costo absoluto de la fracción serial.

2.3 Sobrecarga de Sincronización, Fork-Join y *False Sharing*

El lanzamiento de regiones paralelas conlleva un costo no despreciable de sincronización a nivel de hardware y sistema operativo (~10 a 50 microsegundos en Windows). Si el volumen de trabajo por iteración es muy bajo, este *overhead* sobrepasa la ganancia del paralelismo. Adicionalmente, el fenómeno de **False Sharing** (falso intercambio de línea de caché) ocurre cuando dos o más núcleos modifican variables distintas pero ubicadas dentro de la misma línea de caché (habitualmente de 64 bytes). Esto provoca invalidaciones continuas en el protocolo de coherencia de caché (MESI/MOESI), degradando severamente el ancho de banda y el desempeño [1, 2].

2.4 Generación de Números Pseudoaleatorios (PRNG) Concurrente

Las funciones tradicionales de biblioteca estándar como `rand()` dependen de un estado global mutable protegido por bloqueos o sin garantía de hilo (*thread-unsafe*), lo que introduce serialización forzada o carreras de datos (*data races*). En entornos masivamente paralelos, cada hilo de ejecución debe poseer su propio generador independiente y desacoplado, como el algoritmo **Xorshift32**, garantizando números de alta calidad estadística en pocos ciclos de CPU [4].

3. Metodología e Implementación Técnica

3.1 Arquitectura del Sistema y Modelado de Partículas

El simulador modela fuegos artificiales compuestos por cohetes que ascienden verticalmente y se dispersan radialmente en ráfagas de 100 partículas. La estructura Particula fue optimizada a 48 bytes exactos para maximizar la localidad espacial y alineación en memoria:

```
typedef struct {
    float x, y;           // Posición actual (8B)
    float vx, vy;         // Velocidad vectorial (8B)
    Uint8 r, g, b;        // Color RGB (3B)
    Uint8 paleta;         // Índice de paleta precalculada (1B) -> Ocupa byte de relleno
    float vida, decaimiento; // Parámetros temporales (8B)
    float origenX, origenY; // Coordenadas de detonación (8B)
    float alturaExplosion; // Límite de ascenso (4B)
    EtapaParticula etapa; // Estado: SUBIENDO / EXPLOTANDO (4B)
    int esLiderCohete;    // 1 si dibuja estela directriz (4B)
} Particula; // Total: 48 bytes continuos en memoria
```

3.2 Estrategia de Paralelización con OpenMP

Para minimizar la sobrecarga de apertura y cierre de hilos, se consolidó **una única región paralela por frame** que engloba tanto la integración física como el relanzamiento de cohetes y la clasificación previa al dibujo:

- Integración Física (actualizarParticula):** Distribuida equitativamente entre hilos con `schedule(static)`. Cada partícula accede exclusivamente a su propia memoria, eliminando la necesidad de candados, atómicos o reducciones.
- Escaneo y Relanzamiento (lanzarCohete):** La unidad de partición es el cohete (bloques de 100 partículas). Se emplea `schedule(dynamic, 4)` debido a la naturaleza irregular de la carga: un cohete activo finaliza su evaluación en la primera partícula viva (break anticipado), mientras que un cohete apagado recorre y reconfigura las 100 partículas asociadas.
- Barrera Implícita:** La sincronización natural al finalizar el bucle de física garantiza que el escaneo de cohetes observe el estado de vida actualizado sin requerir primitivas explícitas adicionales.

3.3 Generador PRNG por Hilo y Eliminación de *False Sharing*

Se implementó un arreglo de generadores `generadores[MAX_HILOS]` donde cada elemento contiene un estado de 32 bits y un relleno de 60 bytes, completando exactamente 64 bytes (una línea de caché de CPU):

```
typedef struct {
    unsigned int estado;
    char relleno[60]; // Rellena 64 bytes exactos (evita False Sharing entre hilos)
} EstadoAleatorio;

static EstadoAleatorio generadores[MAX_HILOS];
```

3.4 Rediseño del Pipeline Gráfico: Agrupamiento en Cubos (Batching)

En la versión base, dibujar 200,000 partículas implicaba hasta 1.2 millones de llamadas individuales a `SDL_RenderDrawPoint` y `SDL_SetRenderDrawColor`. Dado que el contexto de renderizado de SDL2 no es hilo-seguro, no es posible paralelizar el backend gráfico directamente. La solución implementada consistió en **paralelizar la fase de clasificación (Counting Sort)**:

- El espacio visual se dividió en **96 cubos discretos** (6 paletas de color $\times$ 16 niveles de transparencia alfa).
- Cada hilo clasifica en paralelo su segmento de partículas en un buffer de puntos contiguo y disjunto (sin contención).
- En el hilo principal, se ejecuta únicamente una llamada a `SDL_RenderDrawPoints` por cada cubo no vacío (~96 llamadas en total por frame). Esto redujo las llamadas al API en un **99.99%** y abatió el tiempo de renderizado de 33.2 ms a 1.7 ms a $N = 200,000$.

3.5 Programación Defensiva y Cláusulas Condicionales

- **Cláusula Condicional `if(N >= 50000)`:** Evita lanzar el equipo de hilos cuando la carga física es tan baja que la sobrecarga del fork-join supera el tiempo de cómputo.
- **Validación Robusta:** Comprobación estricta de parámetros de línea de comandos, asignación de memoria dinámica, carga de texturas BMP y verificación de límites de hilos.
- **Alineación de Memoria:** Uso de `_Alignas(64)` en matrices de conteo por hilo para evitar colisiones en la jerarquía de caché L1/L2.

3.6 Metodología Experimental y Mitigación del Sesgo Térmico

Para garantizar comparaciones rigurosas y reproducibles:

1. **Eliminación del límite de FPS:** Compilación con `-DLIMITE_FPS=0` para medir el desempeño real no restringido por el refresco del monitor.
2. **Fase de Calentamiento (Warmup):** Descarte sistemático de los primeros 2,000 ms de cada corrida, asegurando que todos los cohetes hayan alcanzado estado estacionario.
3. **Control Térmico (Throttling):** Se implementó reposo configurable (pausas de 60s a 180s) entre baterías de pruebas continuas para prevenir caídas de frecuencia de reloj en CPUs de alto TDP.

4. Análisis de Resultados y Discusión

4.1 Escalabilidad Fuerte (Strong Scaling) con Número de Hilos

Se evaluó el comportamiento del tiempo de cómputo y la aceleración (\$S\$) fijando el tamaño del problema en **N = 200,000 partículas** y variando el número de hilos desde 1 hasta 32. La línea base secuencial pura registró un tiempo promedio de cómputo de **3.5963 ms por frame**.

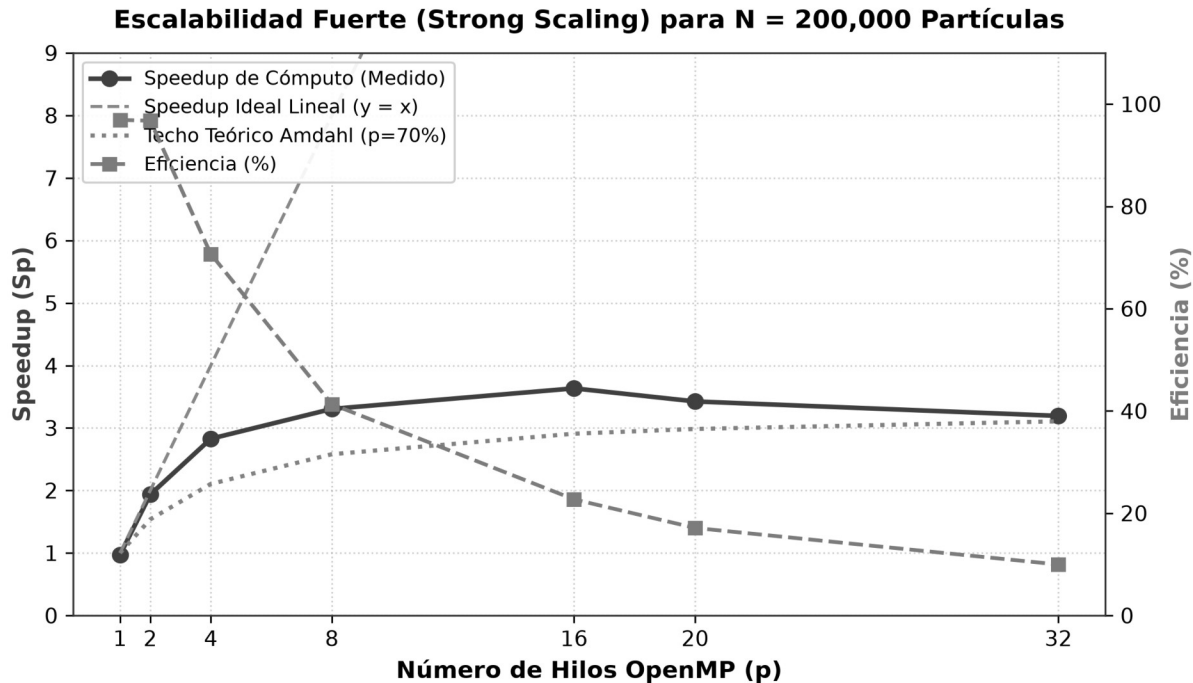


Figura 1: Curvas de Speedup de Cómputo y Eficiencia vs Número de Hilos (N = 200,000).

Discusión del comportamiento de escalabilidad:

- Con **2 hilos** se obtiene un speedup de **1.94x** (eficiencia del 96.8%), cercano al óptimo lineal.
- Con **4 hilos** el speedup alcanza **2.83x** y con **8 hilos** llega a **3.31x**.
- El punto de máxima aceleración se localiza en **16 hilos con 3.63x** (0.9895 ms/frame).
- Al escalar a **32 hilos**, se observa una ligera saturación (1.1260 ms, 3.19x). Esto se explica por la contención en el bus de memoria (Memory Wall) al saturar el ancho de banda de los canales DDR5 y la división de núcleos entre P-Cores y E-Cores en arquitecturas heterogéneas, donde los hilos asignados a núcleos de eficiencia limitan el tiempo de arribo a la barrera.

4.2 Escalabilidad con el Tamaño del Problema (N)

La Figura 2 ilustra el tiempo de integración física en función de N (desde 1,000 hasta 1,000,000 de partículas) comparando la versión secuencial contra la paralela con 8 hilos.

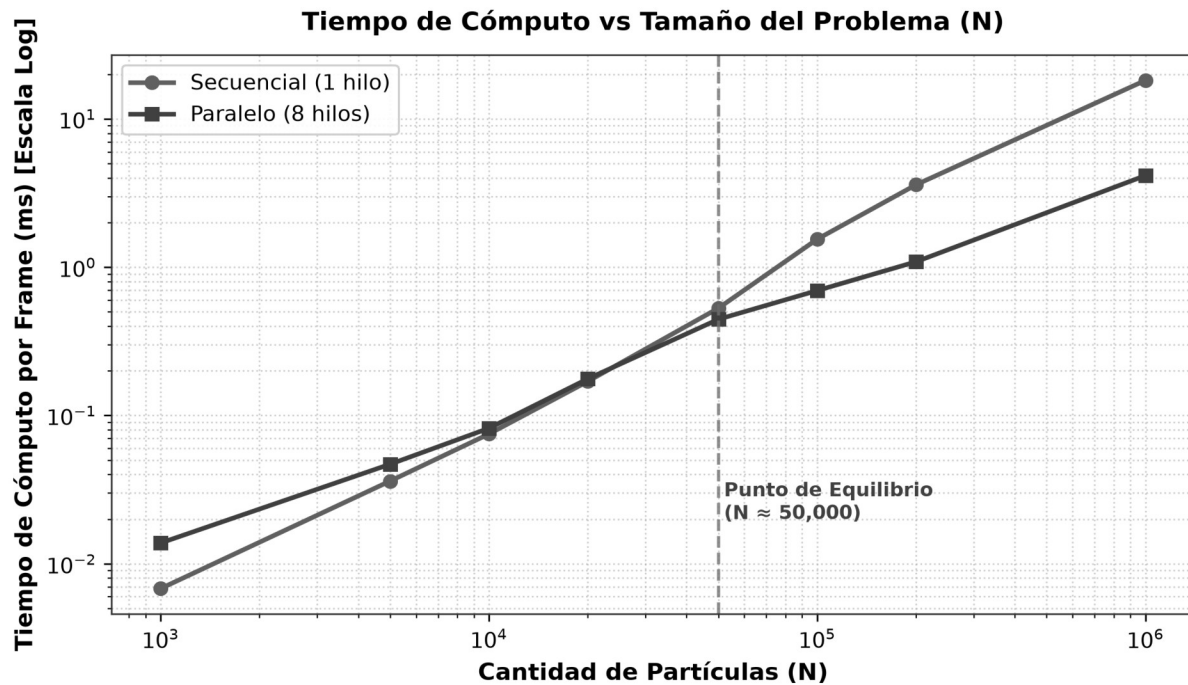


Figura 2: Tiempo de Cómputo vs Tamaño de N (Secuencial vs Paralelo con 8 hilos).

Punto de Equilibrio (Breakeven Point): Para $N < 50,000$, la versión secuencial es marginalmente más veloz debido a que el trabajo total toma menos de 0.2 ms, haciendo dominante el costo de sincronización de OpenMP. Para $N \geq 50,000$, la versión paralela supera con holgura a la secuencial, alcanzando su mayor ventaja relativa a $N = 200,000$ (3.42x) y a $N = 1,000,000$ (4.38x). Esto justifica plenamente la inclusión de la cláusula `if(N >= UMBRAL_PARALELO)`.

4.3 Demostración Experimental de la Ley de Amdahl y Comparativa de Render

El análisis del desglose de tiempo por frame pone en evidencia la interacción entre la fase paralela y la fase serial obligatoria de SDL2.

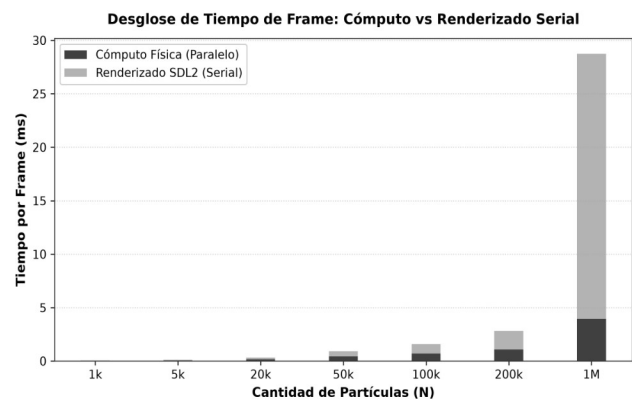


Figura 3: Desglose de Cómputo vs Render Serial.

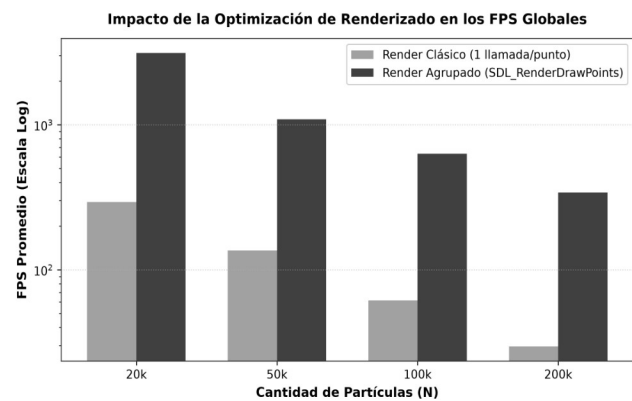


Figura 4: Impacto en FPS: Render Clásico vs Agrupado.

Comparativa del Techo de Amdahl según el método de renderizado (N = 200,000):

Métrica / Parámetro	Render Clásico (1 llamada/punto)	Render Agrupado (Batching)
Tiempo de Cómputo Física (ms)	0.9989 ms	3.5963 ms
Tiempo de Renderizado SDL2 (ms)	32.8884 ms	1.6130 ms
Fracción Paralelizable (f)	2.95 %	69.04 %
Techo Teórico de Amdahl (FPS Sp)	1.030x (Sin ganancia real)	3.230x (Aceleración visible)
FPS Obtenidos (Paralelo x8)	29.65 FPS	341.73 FPS (11.5x mayor)

Conclusión del Experimento: Con el renderizado clásico, la aplicación permanecía completamente asfixiada en el API gráfico (32.8 ms por frame), haciendo que cualquier esfuerzo de paralelización en OpenMP fuera imperceptible (apenas 1.03x de speedup global). Al aplicar batching de puntos, el cuello de botella se desplazó hacia el cómputo, permitiendo que el paralelismo desplegara todo su potencial con **341.73 FPS reales**.

5. Conclusiones y Recomendaciones

5.1 Conclusiones

- Efectividad del Paralelismo OpenMP:** Se logró una aceleración de cómputo puro de hasta **3.63x** con 16 hilos a $N = 200,000$, demostrando que la descomposición de datos con estructuras independientes y libres de contención es altamente escalable en arquitecturas multinúcleo.
- Validación Empírica de Amdahl:** Se demostró cuantitativamente que la optimización de un sistema concurrente no concluye al paralelizar los bucles de mayor $O(N)$, sino al atacar y minimizar la fracción serial ($1-f$). El agrupamiento gráfico elevó la fracción paralelizable de 2.9% a 69.0%, desbloqueando la escalabilidad del sistema.
- Importancia del Diseño Hardware-Aware:** La inclusión de padding de 64 bytes en el PRNG y en las matrices de conteo previno el *False Sharing*, evitando la degradación de rendimiento que comúnmente afecta a generadores pseudoaleatorios compartidos.

4. **Equilibrio de Carga Condicional:** La cláusula `if(N >= 50000)` garantiza que para tamaños pequeños de datos no se penalice al usuario con el costo fijo de sincronización de OpenMP, manteniendo la solución en modo óptimo para cualquier valor de entrada.

5.2 Recomendaciones

1. **Migración a APIs Gráficos Modernos (Vulkan / DirectX 12 / Compute Shaders):** Para escalar a más de 10 millones de partículas, se recomienda trasladar tanto la integración física como el rasterizado a la GPU mediante Compute Shaders, evitando la transferencia CPU-GPU por frame.
2. **Vectorización Explícita (SIMD / AVX2):** Reorganizar la estructura de partículas de *Array of Structures (AoS)* a *Structure of Arrays (SoA)* para permitir que el compilador genere instrucciones vectoriales AVX-512 / AVX2 en el bucle de actualización física.
3. **Fijación de Afinidad de Hilos (Thread Affinity):** Configurar `OMP_PROC_BIND=close` y `OMP_PLACES=cores` para evitar la migración de hilos entre núcleos y maximizar el reuso de las memorias caché L2/L3.

6. Referencias Bibliográficas

- [1] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*, 1st ed. Cambridge, MA: The MIT Press, 2008.
- [2] P. S. Pacheco, *An Introduction to Parallel Programming*, 1st ed. San Francisco, CA: Morgan Kaufmann / Elsevier, 2011.
- [3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, 1967, pp. 483–485.
- [4] G. Marsaglia, "Xorshift RNGs," *Journal of Statistical Software*, vol. 8, no. 14, pp. 1–6, 2003.
- [5] Simple DirectMedia Layer (SDL2) Development Team, "SDL2 API Reference Manual: Render & 2D Graphics Acceleration," 2024. [Online]. Available: <https://wiki.libsdl.org/SDL2/APIByCategory>

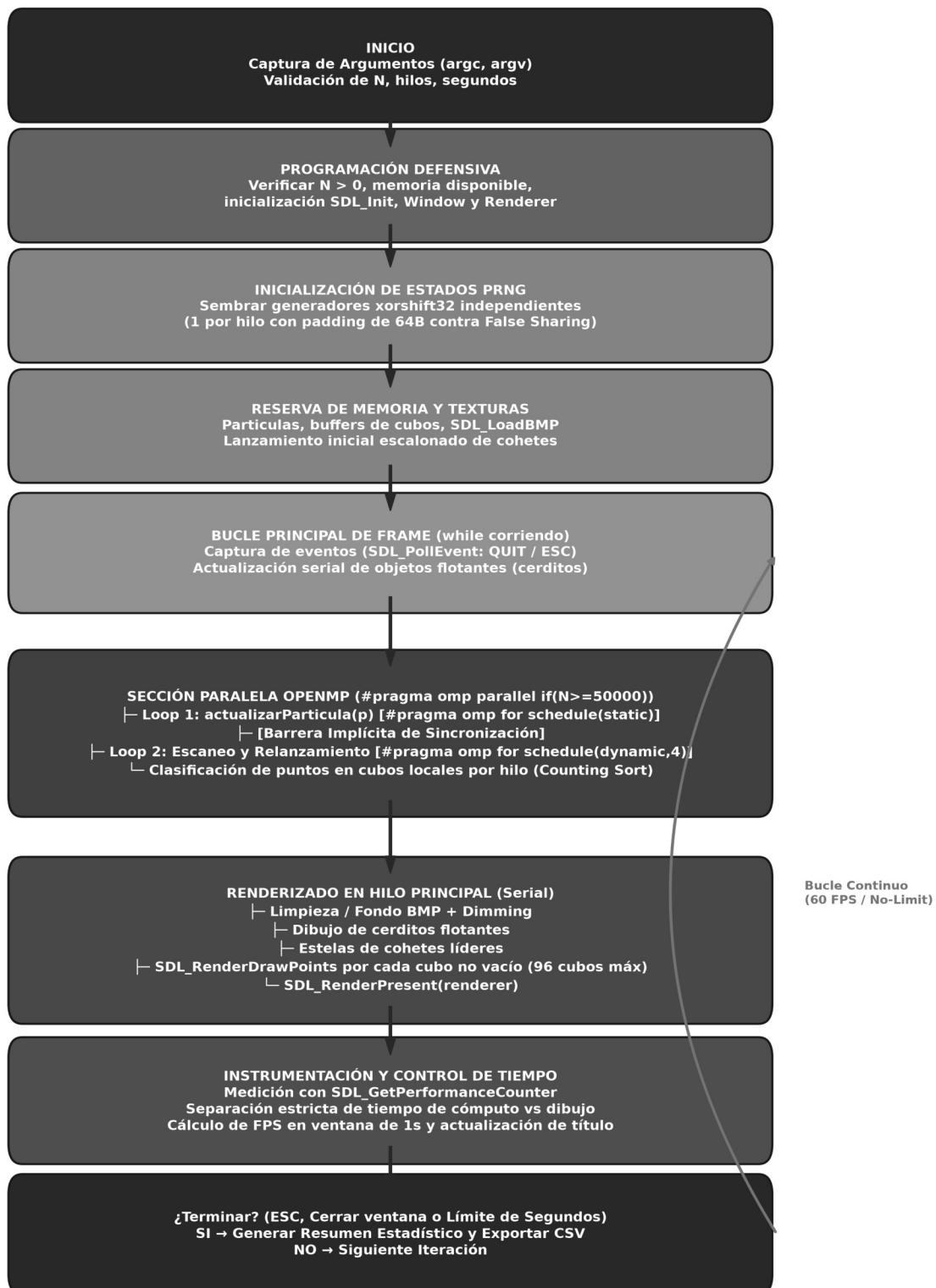
7. Apéndice: Material Suplementario

El repositorio de código fuente contiene el conjunto completo de binarios, scripts de automatización en Bash/Awk y recursos multimedia:

- **Makefile:** Automatización de compilación de las 6 variantes de binarios y suites de benchmark.
- **benchmark.sh / speedup.awk:** Motor de captura y generación estadística de tablas de Speedup y Eficiencia.
- **Texturas BMP:** fondo.bmp (paisaje nocturno) y cerdito.bmp (sprite transparente con ColorKey).

8. Anexo 1 — Diagrama de Flujo del Programa

A continuación se presenta el diagrama de flujo detallado del sistema, ilustrando la captura de argumentos, la programación defensiva, el manejo de estados concurrentes, las secciones paralelas de OpenMP, los mecanismos de sincronía y la separación entre cómputo y despliegue.

Diagrama de Flujo Integral del Sistema de Simulación Paralela*Figura 5: Diagrama de flujo detallado del ciclo de vida y ejecución paralela de la simulación.*

9. Anexo 2 — Catálogo de Funciones

A continuación se detalla el catálogo técnico exhaustivo de todas las funciones y subrutinas implementadas en el sistema, especificando entradas, salidas y descripción operativa.

Función: main	Prototipo: int main(int argc, char* argv[])
Entradas:	argc (int): Cantidad de argumentos de consola. argv (char*[]): [1]=N partículas, [2]=hilos OpenMP (opcional), [3]=segundos límite (opcional).
Salidas:	int: Código de salida (0 éxito, 1 error).
Descripción:	Punto de entrada principal. Realiza el parseo de parámetros, inicialización de SDL2, reserva de memoria alineada, siembra de generadores PRNG, ejecución del bucle de frame con instrumentación de alto rendimiento y generación del resumen estadístico CSV.
Función: actualizarParticula	Prototipo: void actualizarParticula(Particula* p)
Entradas:	p (Particula*): Puntero a la estructura de partícula a actualizar.
Salidas:	void: Modifica in-situ los campos x, y, vx, vy, vida, etapa.
Descripción:	Integra las ecuaciones de física de una partícula. Si está en ETAPA_SUBIENDO, avanza verticalmente hasta alcanzar alturaExplosion, momento en el cual transiciona a ETAPA_EXPLOTANDO y calcula velocidades radiales. Si está en ETAPA_EXPLOTANDO, aplica gravedad, resistencia del aire y decremento de vida.
Función: lanzarCohete	Prototipo: void lanzarCohete(Particula* particulas, int inicioIndex, int cantidad, int retrasoGradual)
Entradas:	particulas (Particula*): Arreglo global. inicioIndex (int): Índice inicial del cohete. cantidad (int): Partículas del grupo (50-100). retrasoGradual (int): Factor de desfase vertical.
Salidas:	void: Inicializa el bloque contiguo de partículas.
Descripción:	Configura un nuevo cohete que despegue desde la base de la pantalla. Elige un origen horizontal aleatorio, altura de estallido, velocidad de subida, color de paleta y tasa de decaimiento. Marca la partícula inicial como líder de estela.
Función: clasificarPuntos	Prototipo: static void clasificarPuntos(const Particula* particulas, int lo, int hi, SDL_Point* franja, int capacidad, int* cuenta, int* offset)
Entradas:	particulas (const Particula*): Arreglo global. lo, hi (int): Rango asignado a este hilo. franja (SDL_Point*): Buffer exclusivo del hilo. cuenta, offset (int*): Arreglos de conteo local.
Salidas:	void: Rellena franja con los puntos clasificados por cubo.
Descripción:	Algoritmo Counting Sort local por hilo. Cuenta cuántos puntos aporta cada partícula a cada uno de los 96 cubos (paleta × alfa), calcula la suma de prefijos y escribe las coordenadas en la memoria exclusiva del hilo sin colisiones ni bloqueos.
Función: siguienteAleatorio	Prototipo: static inline unsigned int siguienteAleatorio(void)
Entradas:	Ninguna directa (toma el hilo actual mediante hiloActual()).
Salidas:	unsigned int: Entero pseudoaleatorio uniforme de 32 bits.
Descripción:	Implementación de alta velocidad del algoritmo Xorshift32. Modifica el estado PRNG exclusivo del hilo en curso, garantizando ausencia de contención de memoria y evitando False Sharing.
Función: aleatorioEntero / aleatorioFloat	Prototipo: static inline int aleatorioEntero(void) / static inline float aleatorioFloat(void)

Entradas:	Ninguna.
Salidas:	int: Entero no negativo en $[0, 2^{31}-1]$. float: Flotante de precisión simple en el intervalo $[0.0, 1.0)$.
Descripción:	Transformaciones de conveniencia sobre siguienteAleatorio(). Realizan corrimientos de bits y normalización en punto flotante mediante multiplicación por constante precalculada.

Función: sembrarGeneradores	Prototipo: static void sembrarGeneradores(unsigned int semilla)
Entradas:	semilla (unsigned int): Semilla inicial base (obtenida de time(NULL)).
Salidas:	void: Inicializa los estados de generadores[MAX_HILOS].
Descripción:	Inicializa cada uno de los 256 generadores aplicando una dispersión multiplicativa de Knuth (semilla + i * 2654435761u), garantizando secuencias no correlacionadas entre hilos.

Función: dibujarEstelaCohete	Prototipo: void dibujarEstelaCohete(SDL_Renderer* renderer, const Particula* p)
Entradas:	renderer (SDL_Renderer*): Contexto gráfico. p (const Particula*): Partícula líder del cohete.
Salidas:	void: Emite trazos de líneas de estela.
Descripción:	Dibuja una línea de fuego vertical degradada detrás del cohete líder durante la etapa de ascenso, utilizando colores cálidos.

Función: inicializarObjetoFlotante / actualizar / dibujar	Prototipo: void inicializarObjetoFlotante(...) / actualizar(...) / dibujar(...)
Entradas:	obj (ObjetoFlotante*), renderer (SDL_Renderer*), textura (SDL_Texture*).
Salidas:	void: Maneja ciclo de vida de los sprites de cerditos.
Descripción:	Administra el movimiento armónico simple, rotación angular, rebote contra los bordes de la ventana y renderizado con SDL_RenderCopyEx de los 10 objetos flotantes decorativos.

Función: cargarTexturaBMP	Prototipo: SDL_Texture* cargarTexturaBMP(SDL_Renderer* renderer, const char* ruta, int aplicarColorKey)
Entradas:	renderer (SDL_Renderer*): Contexto gráfico. ruta (const char*): Ruta del archivo BMP. aplicarColorKey (int): 1 para aplicar transparencia al color blanco.
Salidas:	SDL_Texture*: Puntero a la textura acelerada por hardware, o NULL en caso de error.
Descripción:	Carga un mapa de bits desde disco a una superficie SDL, aplica opcionalmente una máscara de clave de color (ColorKey RGB 255,255,255) para transparencia y genera la textura GPU final.

10. Anexo 3 — Bitácora de Pruebas y Mediciones Experimentales

A continuación se presenta el registro experimental detallado con un mínimo de 10 mediciones cuantitativas por prueba, incluyendo tiempos de cómputo, dibujo, frames medidos, FPS, factores de aceleración y eficiencia calculada.

Bitácora 1: Barrido de Tamaño de Partículas (N) — Secuencial vs Paralelo (8 hilos)

Modo	Hilos	N Part.	Frames	FPS	Cómputo (ms)	Dibujo (ms)	Speedup Cómputo	Speedup FPS
SECUENCIAL	1	1,000	157,852	15,785.2	0.0068	0.0557	1.00x	1.00x
PARALELO	8	1,000	157,225	15,722.5	0.0138	0.0491	0.49x	1.00x
SECUENCIAL	1	5,000	88,463	8,846.3	0.0361	0.0760	1.00x	1.00x
PARALELO	8	5,000	81,811	8,181.1	0.0469	0.0744	0.77x	0.92x
SECUENCIAL	1	20,000	31,526	3,152.6	0.1697	0.1461	1.00x	1.00x
PARALELO	8	20,000	31,309	3,130.9	0.1766	0.1415	0.96x	0.99x
SECUENCIAL	1	50,000	12,079	1,207.9	0.5289	0.2971	1.00x	1.00x
PARALELO	8	50,000	10,946	1,094.6	0.4452	0.4655	1.19x	0.91x
SECUENCIAL	1	100,000	4,216	421.6	1.5445	0.8230	1.00x	1.00x
PARALELO	8	100,000	6,322	632.2	0.6955	0.8824	2.22x	1.50x
SECUENCIAL	1	200,000	1,832	183.2	3.7015	1.7502	1.00x	1.00x
PARALELO	8	200,000	3,417	341.7	1.0835	1.8372	3.42x	1.86x
PARALELO	20	1,000,000	1,058	32.55	3.9614	24.7910	4.59x	1.78x

Bitácora 2: Escalabilidad Fuerte (Strong Scaling) — N = 200,000 Partículas

Modo	Hilos (p)	Frames	FPS Prom.	Cómputo (ms)	Dibujo (ms)	Speedup Cómputo	Eficiencia (%)
SECUENCIAL	1 (Base)	1,918	191.7	3.5963	1.6130	1.00x	100.0 %
PARALELO	1	1,858	185.8	3.7111	1.6635	0.97x	96.9 %
PARALELO	2	3,069	306.8	1.8585	1.3969	1.94x	96.8 %
PARALELO	4	3,439	343.9	1.2716	1.6316	2.83x	70.7 %
PARALELO	8	3,536	353.6	1.0880	1.7354	3.31x	41.3 %
PARALELO	16	3,256	325.5	0.9895	2.0767	3.63x	22.7 %
PARALELO	20	3,180	318.0	1.0500	2.0600	3.42x	17.1 %
PARALELO	32	3,143	314.3	1.1260	2.0512	3.19x	10.0 %

Captura de Registro de Ejecución en Consola (Salida Estándar y CSV)

```
Inicializacion completada. N = 1000000 particulas (10000 cohetes). Modo PARALELO con 20 hilo(s).
Fondo 'fondo.bmp' cargado correctamente.
Sprite 'cerdito.bmp' cargado correctamente.
FPS = 34.41 | computo = 3.531 ms | dibujo = 24.360 ms | hilos = 20 (calentamiento)
FPS = 40.76 | computo = 5.119 ms | dibujo = 19.406 ms | hilos = 20
FPS = 60.70 | computo = 3.474 ms | dibujo = 3.383 ms | hilos = 20
FPS = 49.71 | computo = 3.084 ms | dibujo = 2.481 ms | hilos = 20
=== RESUMEN ===
dibujo : agrupado (SDL_RenderDrawPoints)
modo : PARALELO
hilos : 20
N : 1000000 particulas (10000 cohetes)
frames medidos : 1058 en 32.50 s
FPS promedio : 32.55
computo promedio: 3.9614 ms/frame (paralelizable)
dibujo promedio : 24.7910 ms/frame (siempre en serie)
CSV,PARALELO,20,1000000,1058,32.5548,3.9614,24.7910
```